



Tecnológico de Monterrey

Sebastián Mansilla Cots

A01712805

14/03/2026

TC2005B

Lab 20: Consultas SQL

Consulta de un tabla completa

Algebra relacional.

materiales

Sentencia (SQL):

```
select * from materiales
```

Salida de la consulta:

clave	descripcion	precio	impuesto
1000	Varilla 3/16	100	10
1010	Varilla 4/32	115	11.5
1020	Varilla 3/17	130	13

Total de renglones: 45

Selección

Algebra relacional.

$SL\{clave=1000\}(materiales)$

Sentencia (SQL):

```
select * from materiales
```

```
where clave=1000
```

Salida de la consulta:

clave	descripcion	precio	impuesto
1000	Varilla 3/16	100	10

Total de renglones: 1

Proyección

Algebra relacional.

$PR\{clave, rfc, fecha\}(entregan)$

Sentencia (SQL):

select clave,rfc,fecha from entregan

Salida de la consulta:

clave	rfc	fecha
1000	AAAA800101	2001-12-13
1000	AAAA800101	1999-07-13
1010	BBBB800101	1998-07-28

Total de renglones: 87

Reunión Natural

Algebra relacional.

entregan JN materiales

Sentencia (SQL):

select * from materiales,entregan

where materiales.clave = entregan.clave

Salida de la consulta:

clave	descripcion	precio	impuesto	clave	rfc	numero	fecha	cantidad
1000	Varilla 3/16	100	10	1000	AAAA800101	5000	2001-12-13	165
1000	Varilla 3/16	100	10	1000	AAAA800101	5019	1999-07-13	254
1010	Varilla 4/32	115	11.5	1010	BBBB800101	5001	1998-07-28	528

Total de renglones: 87

Si algún material no ha se ha entregado ¿Aparecería en el resultado de esta consulta?

No ya que si uno material fue entregado debe tener su clave en la tabla de entregan, y como en el JOIN igualamos las claves de ambas tablas solo aparecen aquellos que están en las dos.

Reunión con criterio específico

Algebra relacional.

entregan JN{entregan.numero <= proyectos.numero} proyectos

Sentencia (SQL):

```
select * from entregan,proyectos
```

```
where entregan.numero <= proyectos.numero
```

Salida de la consulta:

clave	rfc	numero	fecha	cantidad	numero	denominacion
1000	AAAA800101	5000	2001-12-13	165	5000	Vamos Mexico
1200	EEEE800101	5000	2003-03-15	177	5000	Vamos Mexico
1400	AAAA800101	5000	1999-04-07	382	5000	Vamos Mexico

Total de renglones: 87

Unión (se ilustra junto con selección)

Algebra relacional.

SL{clave=1450}(entregan) UN SL{clave=1300}(entregan)

Sentencia (SQL):

```
(select * from entregan where clave=1450)
```

union

```
(select * from entregan where clave=1300)
```

Salida de la consulta:

clave	rfc	numero	fecha	cantidad
1300	GGGG800101	5005	2004-02-28	521
1300	GGGG800101	5010	2001-02-10	119

Total de renglones = 2

¿Cuál sería una consulta que obtuviera el mismo resultado sin usar el operador Unión? Compruébalo.

La consulta sería: SELECT * FROM entregan WHERE clave=1450 OR clave=1300;

Y el resultado es el siguiente:

clave	rfc	numero	fecha	cantidad
1300	GGGG800101	5005	2004-02-28	521
1300	GGGG800101	5010	2001-02-10	119

Como se puede observar da el mismo resultado sin necesitar de un UNION

Intersección (se ilustra junto con selección y proyección)

Algebra relacional.

$PR\{clave\}(SL\{numero=5001\}(entregan)) \cap$

$PR\{clave\}(SL\{numero=5018\}(entregan))$

Sentencia (SQL):

$(select\ clave\ from\ entregan\ where\ numero=5001)$

$intersect$

$(select\ clave\ from\ entregan\ where\ numero=5018)$

Salida de la consulta:

clave

1010

Total de renglones: 1

Diferencia (se ilustra con selección)

Algebra relacional.

$entregan - SL\{clave=1000\}(entregan)$

Sentencia (SQL):

$(select\ * \ from\ entregan)$

$minus$

(select * from entregan where clave=1000)

Sentencia real (SQL):

SELECT * FROM entregan WHERE clave NOT IN(SELECT clave FROM entregan WHERE clave=1000);

Salida de la consulta:

clave	rfc	numero	fecha	cantidad
1010	BBBB800101	5001	1998-07-28	528
1010	BBBB800101	5018	1997-02-09	523
1020	CCCC800101	5002	2003-12-16	582

Total de renglones: 85

Producto cartesiano

Algebra relacional.

entregan X materiales

Sentencia (SQL):

select * from entregan,materiales

Salida de la consulta:

clave	rfc	numero	fecha	cantidad	clave	descripcion	precio	impuesto
1000	AAAA800101	5000	2001-12-13	165	1000	Varilla 3/16	100	10
1000	AAAA800101	5000	2001-12-13	165	1010	Varilla 4/32	115	11.5
1000	AAAA800101	5000	2001-12-13	165	1020	Varilla 3/17	130	13

Total de renglones: 87

¿Cómo está definido el número de tuplas de este resultado en términos del número de tuplas de entregan y de materiales?

Se hace una combinación entre cada tupla de entregan con las de materiales.

Construcción de consultas a partir de una especificación

Plantea ahora una consulta para obtener las descripciones de los materiales entregados en el año 2000.

Consulta:

```
SELECT m.descripcion FROM materiales m JOIN entregan e ON m.clave = e.clave WHERE fecha BETWEEN '01/01/00' AND '31/12/00';
```

Salida de la consulta:

descripcion

Varilla 3/16

Varilla 3/17

Varilla 4/33

Total de renglones: 47

¿Por qué aparecen varias veces algunas descripciones de material?

Porque ese material se entrego varias veces en ese año.

Uso del calificador distinct

En el resultado anterior, observamos que una misma descripción de material aparece varias veces.

Agrega la palabra distinct inmediatamente después de la palabra select a la consulta que planteaste antes.

¿Qué resultado obtienes en esta ocasión?

En este caso ya no aparecen materiales repetidos, bajando el total de renglones de 47 a 31.

Ordenamientos.

Obtén los números y denominaciones de los proyectos con las fechas y cantidades de sus entregas, ordenadas por número de proyecto, presentando las fechas de la más reciente a la más antigua.

Consulta:

```
SELECT p.numero, p.denominacion, e.fecha, e.cantidad FROM proyectos p JOIN  
entregan e ON p.numero = e.numero GROUP BY p.numero ORDER BY `e`.`fecha`  
DESC
```

Resultado de consulta:

numero	denominacion	fecha ▾ 1	cantidad
5007	Construcción de plaza Magnolias	2006-11-21	2
5016	Ampliación de la carretera a la huasteca	2005-06-07	295
5008	Televisa en acción	2005-04-03	86

Total de renglones: 20

Uso de expresiones.

En álgebra relacional los argumentos de una proyección deben ser columnas. Sin embargo en una sentencia SELECT es posible incluir expresiones aritméticas o funciones que usen como argumentos de las columnas de las tablas involucradas o bien constantes. Los operadores son:

+ Suma

- Resta

* Producto

/ División

Las columnas con expresiones pueden renombrarse escribiendo después de la expresión un alias que puede ser un nombre arbitrario; si el alias contiene caracteres que no sean números o letras (espacios, puntos etc.) debe encerrarse entre comillas dobles (" nuevo nombre"). Para SQL Server también pueden utilizarse comillas simples.

Operadores de cadena

```
SELECT * FROM productos where Descripcion LIKE 'Si%'
```

Consulta real:

```
SELECT * FROM materiales m where m.descripcion LIKE 'Si%';
```


¿Qué resultado obtienes?

Obtuve todos los materiales que tienen en su descripción algo que inicia en Si y le sigue una cadena de tipo string.

Explica que hace el símbolo '%'.

El símbolo % define que lo que le interesa es buscar algo que inicie con un valor N% seguido de cualquier cosa, sin importar la cantidad de caracteres que hay después o antes (en caso de ponerse %N).

¿Qué sucede si la consulta fuera : LIKE 'Si' ?

¿Qué resultado obtienes?

Va a buscar materiales que tengan la descripción de “Si” pero solo esos dos caracteres ninguna otra cosa. Esto genera que no haya resultado ya que no hay ninguna descripción que solo tenga “Si”.

Otro operador de cadenas es el de concatenación, (+, +=) este operador concatena dos o más cadenas de caracteres.

Su sintaxis es : Expresión + Expresión.

Un ejemplo de su uso, puede ser:

```
SELECT (Apellido + ', ' + Nombre) as Nombre FROM Personas;
```

```
DECLARE @foo varchar(40);
```

```
DECLARE @bar varchar(40);
```

```
SET @foo = '¿Que resultado';
```

```
SET @bar = ' ¿¿¿??? '
```

```
SET @foo += ' obtienes?';
```

```
PRINT @foo + @bar;
```

¿Qué resultado obtienes de ejecutar el siguiente código?

Resultado

¿Que resultado obtienes? ¿¿¿???

¿Para qué sirve DECLARE?

Sirve para declarar una variable local

¿Cuál es la función de @foo?

Al usar @ se define que lo que le sigue es una variable definida por el usuario y no una tabla

¿Que realiza el operador SET?

Es lo que se usa para declarar una asignación su función es meter un valor dentro de una variable.

Sin embargo, tenemos otros operadores como [] , [^] y _.

[] - Busca coincidencia dentro de un intervalo o conjunto dado. Estos caracteres se pueden utilizar para buscar coincidencias de patrones como sucede con LIKE.

[^] - En contra parte, este operador coincide con cualquier caracter que no se encuentre dentro del intervalo o del conjunto especificado.

_ - El operador _ o guion bajo, se utiliza para coincidir con un caracter de una comparación de cadenas.

Ahora explica el comportamiento, función y resultado de cada una de las siguientes consultas:

```
SELECT RFC FROM Entregan WHERE RFC LIKE '[A-D]%';
```

Comportamiento:

Buscara todos los RFC en entregan donde el RFC inicie entre las letras A,B,C o D.

Resultado:

Si se usa la expresión como esta no mostrara nada en MySQL porque el like va a buscar un rfc que inicie con [A-D] por lo que la consulta debe seguir una expresión regular quedando de la siguiente forma: SELECT e.rfc FROM entregan e WHERE e.rfc REGEXP '^[A-D]';

rfc
AAAA800101
AAAA800101
BBBB800101

SELECT RFC FROM Entregan WHERE RFC LIKE '[^A]%';

Comportamiento:

Buscara todos los RFC en entregan donde el RFC no inicie con la letra A.

Resultado:

Misma situación que antes, la consulta sería: SELECT e.rfc FROM entregan e WHERE e.rfc REGEXP '^[^A]';

rfc
BBBB800101
BBBB800101
CCCC800101

SELECT Numero FROM Entregan WHERE Numero LIKE '____6';

Comportamiento:

Busca todos los numeros en entregan que tengan 3 caracteres previos al 6.

Resultado:

Numero
5016
5006
5006

Operadores compuestos.

Los operadores compuestos ejecutan una operación y establecen un valor.

+ = (Suma igual)

- = (Restar igual)

* = (Multiplicar igual)

/ = (Dividir igual)

% = (Módulo igual)

Operadores Lógicos.

Los operadores lógicos comprueban la verdad de una condición, al igual que los operadores de comparación, devuelven un tipo de dato booleano (True, false o unknown).

ALL Es un operador que compara un valor numérico con un conjunto de valores representados por un subquery. La condición es verdadera cuando todo el conjunto cumple la condición.

ANY o SOME Es un operador que compara un valor numérico con un conjunto de valores. La condición es verdadera cuando al menos un dato del conjunto cumple la condición.

La sintaxis para ambos es: valor_numerico {operador de comparación} subquery

BETWEEN Es un operador para especificar intervalos. Una aplicación muy común de dicho operador son intervalos de fechas.

```
SELECT Clave,RFC,Numero,Fecha,Cantidad
FROM Entregan
WHERE Numero Between 5000 and 5010;
```

¿Cómo filtrarías rangos de fechas?

```
SELECT Clave,RFC,Numero,Fecha,Cantidad
FROM Entregan
WHERE Fecha Between 'fecha_inicio' and 'fecha_fin';
```

EXISTS Se utiliza para especificar dentro de una subconsulta la existencia de ciertas filas.

```
SELECT RFC,Cantidad, Fecha,Numero
FROM [Entregan]
WHERE [Numero] Between 5000 and 5010 AND
Exists ( SELECT [RFC]
FROM [Proveedores]
WHERE RazonSocial LIKE 'La%' and [Entregan].[RFC] = [Proveedores].[RFC] )
```

¿Qué hace la consulta?

Muestra el RFC, Cantidad, Fecha y Numero de las entregas con numero entre 5000 y 5010 donde la razón social del proveedor inicie con La.

¿Qué función tiene el paréntesis () después de EXISTS?

Especifica que se va a hacer una subconsulta dentro de la consulta.

IN Especifica si un valor dado tiene coincidencias con algún valor de una subconsulta. NOTA: Se utiliza dentro del WHERE pero debe contener un parametro. Ejemplo: Where proyecto.id IN Lista_de_Proyectos_Subquery

Tomando de base la consulta anterior del EXISTS, realiza el query que devuelva el mismo resultado, pero usando el operador IN

```
SELECT RFC,Cantidad, Fecha,Numero FROM Entregan WHERE Numero  
Between 5000 and 5010 AND RFC IN ( SELECT RFC FROM Proveedores  
WHERE RazonSocial LIKE 'La%' and Entregan.RFC = Proveedores.RFC);
```

NOT Simplemente niega la entrada de un valor booleano.

Tomando de base la consulta anterior del EXISTS, realiza el query que devuelva el mismo resultado, pero usando el operador NOT IN Realiza un ejemplo donde apliques algún operador : ALL, SOME o ANY.

```
SELECT RFC,Cantidad, Fecha,Numero FROM Entregan WHERE Numero  
Between 5000 and 5010 AND RFC NOT IN ( SELECT RFC FROM Proveedores  
WHERE RazonSocial NOT LIKE 'La%' and Entregan.RFC = Proveedores.RFC);
```

Ejemplo con ALL, SOME o ANY:

```
SELECT RFC, Cantidad, Fecha, Numero FROM Entregan WHERE Numero  
BETWEEN 5000 AND 5010 AND RFC > ALL (SELECT RFC FROM Proveedores  
WHERE RazonSocial LIKE 'La%');
```

El Operador TOP, es un operador que recorre la entrada, un query, y sólo devuelve el primer número o porcentaje específico de filas basado en un criterio de ordenación si es posible.

¿Qué hace la siguiente sentencia? Explica por qué.

```
SELECT TOP 2 * FROM Proyectos
```

Esta sentencia regresa las primeras dos tuplas registradas en la tabla Proyectos, esto debido a que se pide que regrese la tupla completa y como no se especificó un criterio de ordenación se toman las primeras dos tuplas registradas. Aunque esto solo funcionaría en SQL server, en MySQL que es el uso yo se tendría que usar LIMIT y este no va al inicio de la consulta, se construye de la siguiente manera:

```
SELECT * FROM proyectos LIMIT 2;
```

¿Qué sucede con la siguiente consulta? Explica por qué.

```
SELECT TOP Numero FROM Proyectos
```

Da error porque sería decir quiero ver el Numero de tuplas registradas, donde Numero es un atributo de la tabla, esto genera un error porque TOP necesita un número fijo de valores que quiere uno ver, y como "Numero" es un atributo que cambia con cada Tupla no sirve para hacer esa consulta.

Modificando la estructura de un tabla existente.

Agrega a la tabla materiales la columna PorcentajImpuesto con la instrucción:

```
ALTER TABLE materiales ADD PorcentajImpuesto NUMERIC(6,2);
```

A fin de que los materiales tengan un impuesto, les asignaremos impuestos ficticios basados en sus claves con la instrucción:

```
UPDATE materiales SET PorcentajImpuesto = 2*clave/1000;
```

esto es, a cada material se le asignará un impuesto igual al doble de su clave dividida entre diez.

Revisa la tabla de materiales para que compruebes lo que hicimos anteriormente.

¿Qué consulta usarías para obtener el importe de las entregas es decir, el total en dinero de lo entregado, basado en la cantidad de la entrega y el precio del material y el impuesto asignado?

```
SELECT E.numero as 'Numero entrega', M.precio, M.impuesto, E.cantidad,  
(M.precio + M.impuesto) * (E.cantidad) as 'Importe de entrega' FROM entregan E  
JOIN materiales M ON E.clave = M.clave;
```

Creación de vistas

La sentencia:

```
Create view nombrevista (nombrecolumna1 , nombrecolumna2 ,...,  
nombrecolumna3 )
```

```
as select...
```

Permite definir una vista. Una vista puede pensarse como una consulta etiquetada con un nombre, ya que en realidad al referirnos a una vista el DBMS realmente ejecuta la consulta asociada a ella, pero por la cerradura del álgebra relacional, una consulta puede ser vista como una nueva relación o tabla, por lo que es perfectamente válido emitir la sentencia:

```
select * from nombrevista
```

¡Como si nombrevista fuera una tabla!

Comprueba lo anterior, creando vistas para cinco de las consultas que planteaste anteriormente en la práctica . Posteriormente revisa cada vista creada para comprobar que devuelve el mismo resultado.

1.

```
CREATE VIEW Vista_Entrega_Importe AS SELECT E.numero as 'Numero entrega', M.precio, M.impuesto, E.cantidad, (M.precio + M.impuesto) * (E.cantidad) as 'Importe de entrega' FROM entregan E JOIN materiales M ON E.clave = M.clave;
```
2.

```
CREATE VIEW vista_entregas_proveedor_la AS SELECT RFC,Cantidad, Fecha,Numero FROM Entregan WHERE Numero Between 5000 and 5010 AND RFC IN ( SELECT RFC FROM Proveedores WHERE RazonSocial LIKE 'La%' and Entregan.RFC = Proveedores.RFC);
```
3.

```
CREATE VIEW vista_num_entrega_xyz6 AS SELECT Numero FROM Entregan WHERE Numero LIKE '____6';
```


4. CREATE VIEW vista_materiales_año2000 AS SELECT m.descripcion
FROM materiales m JOIN entregan e ON m.clave = e.clave WHERE fecha
BETWEEN '2000/01/01' AND '2000/12/31';
5. CREATE VIEW vista_union_clave_1450_1300 AS (select * from entregan
where clave=1450) union (select * from entregan where clave=1300);

A continuación se te dan muchos enunciados de los cuales deberás generar su correspondiente consulta.

En el reporte incluye la sentencia, una muestra de la salida (dos o tres renglones) y el número de renglones que SQL Server reporta al final de la consulta.

Los materiales (clave y descripción) entregados al proyecto "México sin ti no estamos completos".

Consulta:

```
SELECT M.clave, M.descripcion FROM materiales M JOIN entregan E ON M.clave  
= E.clave JOIN proyectos P ON E.numero = P.numero WHERE P.denominacion =  
"México sin ti no estamos completos";
```

Salida:

clave	descripcion
1030	Varilla 4/33
1230	Cemento
1430	Pintura B1022

Total de renglones: 3

Los materiales (clave y descripción) que han sido proporcionados por el proveedor "Acme tools".

Consulta:

```
SELECT M.clave, M.descripcion FROM materiales M JOIN entregan E ON M.clave  
= E.clave JOIN proveedores P ON E.rfc = P.rfc WHERE P.razonsocial = "Acme  
tools";
```

Salida:

Salida vacía porque no hay ningún proveedor “Acme tools”

El RFC de los proveedores que durante el 2000 entregaron en promedio cuando menos 300 materiales.

Consulta:

```
SELECT P.RFC, AVG(E.cantidad), e.fecha FROM proveedores P JOIN entregan E
ON P.rfc = E.rfc WHERE E.fecha BETWEEN '2000/01/01' AND '2000/12/31'
GROUP BY E.cantidad HAVING AVG(E.cantidad) >= 300;
```

Salida:

RFC	AVG(E.cantidad)	fecha
CCCC800101	466.0000	2000-12-07
FFFF800101	562.0000	2000-04-13
GGGG800101	583.0000	2000-06-30

Total de renglones: 5

El Total entregado por cada material en el año 2000.

```
Consulta: SELECT M.clave, M.descripcion, SUM(E.cantidad) AS 'Total' FROM
materiales M JOIN entregan E ON M.clave = E.clave WHERE E.fecha BETWEEN
'2000/01/01' AND '2000/12/31' GROUP BY E.clave;
```

Salida:

clave	descripcion	Total
1020	Varilla 3/17	8
1050	Varilla 4/34	623
1100	Block	466

Total de renglones: 11

La Clave del material más vendido durante el 2001. (se recomienda usar una vista intermedia para su solución)

Consulta: SELECT clave, MAX(Total) FROM vista_total_materiales_2001 V;

Salida:

clave	MAX(Total)
1000	1091

Productos que contienen el patrón 'ub' en su nombre.

Consulta:

SELECT * FROM `materiales` WHERE descripcion LIKE '%ub%';

Salida:

clave	descripcion	precio	impuesto	Porcentajelmpuesto
1180	Recubrimiento P1001	200	20	2.36
1190	Recubrimiento P1010	220	22	2.38
1200	Recubrimiento P1019	240	24	2.40

Total de renglones: 12

Denominación y suma del total a pagar para todos los proyectos.

Consulta: SELECT P.denominacion, SUM(V.`Importe de entrega`) AS 'Total a pagar proyecto' FROM vista_entrega_importe V JOIN entregan E ON E.numero = V.`Numero entrega` JOIN proyectos P ON P.numero = E.numero GROUP BY P.denominacion;

Salida:

denominacion	Total a pagar proyecto
Ampliación de la carretera a la huasteca	3108242.5
Aztecon	483763.5
CIT Campeche	520591.5

Total de renglones: 20

Denominación, RFC y RazonSocial de los proveedores que se suministran materiales al proyecto Televisa en acción que no se encuentran apoyando al proyecto Educando en Coahuila (Solo usando vistas).

Consultas:

Vista 1: CREATE VIEW vista_proveedores_televisa AS SELECT Pr.denominacion, P.rfc, P.razonsocial FROM proveedores P JOIN entregan E ON E.rfc = P.rfc JOIN proyectos Pr ON E.numero = Pr.numero WHERE Pr.denominacion = 'Televisa en acción';

Vista 2: CREATE VIEW vista_proveedor_educando AS SELECT Pr.denominacion, P.rfc, P.razonsocial FROM proveedores P JOIN entregan E ON E.rfc = P.rfc JOIN proyectos Pr ON E.numero = Pr.numero WHERE Pr.denominacion = 'Educando en Coahuila';

(SELECT denominacion,rfc,razonsocial FROM vista_proveedores_televisa) except (SELECT denominacion,rfc,razonsocial FROM vista_proveedor_educando);

Salida:

denominacion	rfc	razonsocial
Televisa en acción	AAAA800101	La fragua
Televisa en acción	CCCC800101	La Ferre
Televisa en acción	DDDD800101	Cecoferre

Denominación, RFC y RazonSocial de los proveedores que se suministran materiales al proyecto Televisa en acción que no se encuentran apoyando al proyecto Educando en Coahuila (Sin usar vistas, utiliza not in, in o exists).

Consulta: SELECT denominacion,P.rfc,razonsocial FROM proveedores P JOIN entregan E ON P.rfc = E.rfc JOIN proyectos Pr ON Pr.numero = E.numero WHERE denominacion = "Televisa en acción" AND razonsocial NOT IN(SELECT razonsocial FROM proveedores P JOIN entregan E ON P.rfc = E.rfc JOIN proyectos Pr ON Pr.numero = E.numero WHERE denominacion = "Educando en Coahuila");

Salida:

denominacion	rfc	razonsocial
Televisa en acción	CCCC800101	La Ferre
Televisa en acción	DDDD800101	Cecoferre
Televisa en acción	DDDD800101	Cecoferre

Total de renglones: 3

Costo de los materiales y los Materiales que son entregados al proyecto Televisa en acción cuyos proveedores también suministran materiales al proyecto Educando en Coahuila.

Consulta: SELECT M.descripcion, M.precio FROM materiales M JOIN entregan E on M.clave = e.clave JOIN proyectos P ON P.numero = E.numero WHERE P.denominacion = "Televisa en acción" AND E.rfc IN (SELECT E.rfc FROM entregan E JOIN proyectos P ON P.numero = E.numero WHERE P.denominacion = "Educando en Coahuila");

Salida:

descripcion	precio
Ladrillos rojos	50
Tepetate	34

Total de renglones: 2

Reto: Usa solo el operador NOT IN en la consulta anterior (No es parte de la entrega).

Nombre del material, cantidad de veces entregados y total del costo de dichas entregas por material de todos los proyectos.

Consulta:

SELECT M.descripcion,COUNT(M.clave), SUM((M.precio + M.impuesto) * (E.cantidad)) as 'Importe de entrega' FROM materiales M JOIN entregan E ON M.clave = E.clave GROUP BY M.clave, M.descripcion;

Salida:

descripcion	COUNT(M.clave)	Importe de entrega
Varilla 3/16	2	46090
Varilla 4/32	2	132951.5
Varilla 3/17	2	84370

Total de renglones: 44